MAN-40389 Advanced Data Analytics and Machine Learning

Individual report

(Student ID:25036701)

Table of Contents

Introduction **3**

Data preparation **3**

Exploratory Data Analysis **4**

ML Model Fitting and Scoring **5**

Cross validation **5**

Hyperparameter Optimization **6**

Model Comparison **7**

References **9**

Appendix (Code) **10**

Introduction

In this proposed project, a machine learning algorithm will be developed to determine the most economic power plant for the forecasted demand to produce electricity by optimizing the generation cost of different plants for consideration.

The tool employs a set of datasets with integrated demand characteristics, plant information, and observed generation costs, enabling a detailed evaluation of how plants respond differently to different conditions of operation. The performance of the model is assessed using a formula for error (Formula 2) and a root mean square error measure (Formula 3), providing a set of measures that allow for the accuracy of forecasts to be precisely assessed considering actual forecasts of generation costs. The process of this report follows a logical sequence, from preparing the data to exploring variables to developing a modelling tool (Raza, Moazeni and Khazaei, 2020).

Data preparation

In the data preparation stage, the process was organized around generation cost because this is the main target variable, and it directly reflects the project's economic objective of identifying the most economical plant. Data preparation is about cleaning, structuring, and transforming the raw data so that the key outcome can be modelled as accurately as possible. The focus, therefore, was directed to make the generation cost variable complete, consistent, and well aligned with its main drivers, such as relevant plant characteristics and demand indicators. By centring the preparation steps on features that serve to explain generation cost, the dataset stays tightly aligned with the cost forecasting task and avoids unnecessary work on variables that do not contribute to the final objective; this keeps the pipeline simpler and more robust. This approach follows standard practice in cost modelling and cost forecasting projects, where preprocessing is explicitly organized around producing a reliable target cost measure along with the most important explanatory variables, so that the resulting models support clear, defensible economic decisions.

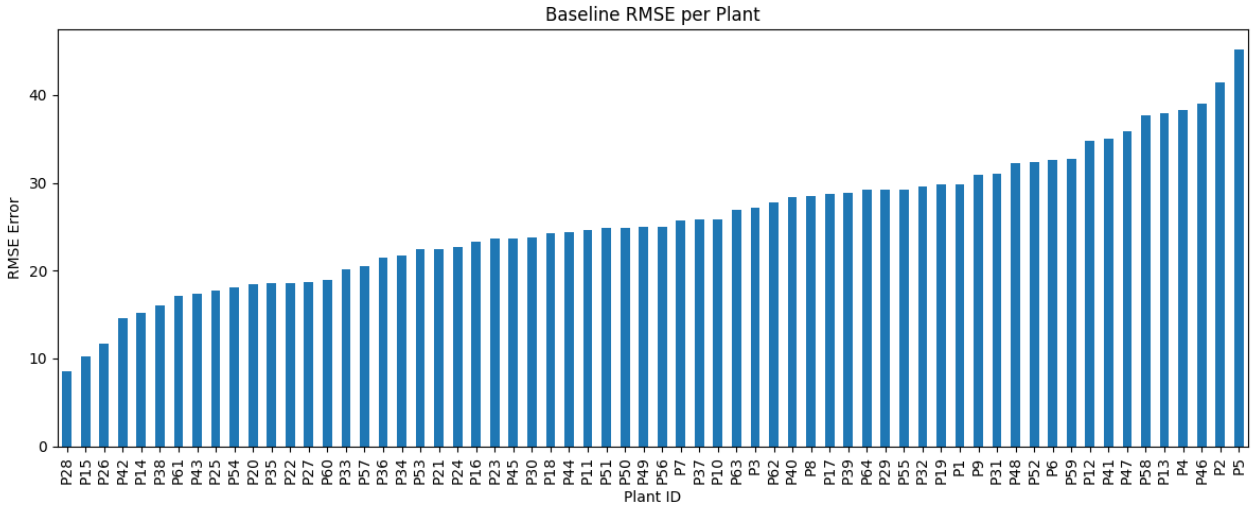


Fig 1. Per plant Baseline RMSE

Interpretation: This script, 1_preparation.py, implements the first stage of data preparation, cleaning, and optimization necessary for a machine learning pipeline, according to Task Q1. It commences with the loading of three major datasets: demand.csv, plants.csv, and generation_costs.csv, and promptly drops all rows in the generation_costs file that do not

include the target variable, Cost_USD_per_MWh. These three datasets are then merged together through left-joining on the shared keys "Demand ID" and "Plant ID" into one integrated dataframe.

To handle missing feature values, the script implements a coherent imputation strategy. Numeric columns whose column names start with "DF" or "PF" fill in their missing values with the median of that column. Categorical fields like "Plant Type", "DF_region", and similar variables replace their missing values with the label "Unknown", as that retains all records but keeps the information consistent and usable.

Next, the script calculates a Baseline RMSE per plant. First, it determines the minimum actual cost for each “Demand ID” (BestCost), then it calculates the accumulated RMSE-Eq. for each plant, depending on how far its cost is from that BestCost. These baseline RMSEs are plotted and saved as baseline_plant_rmse.png and also used for simple feature selection step - it removes 20 worst performing plants, i.e., having the highest baseline RMSE, from the dataset. Finally, the cleaned and pruned dataframe is saved as cleaned_merged_data.csv, which concludes Step 1 of the workflow (Raza, Moazeni and Khazaei, 2020).

Exploratory Data Analysis

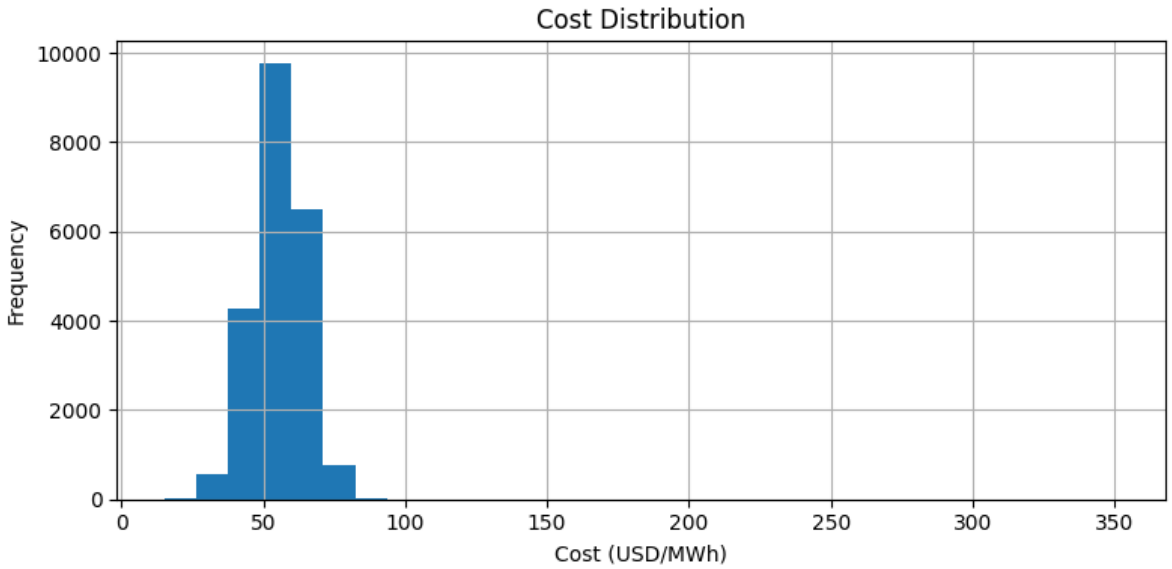


Fig.2 Cost Distribution

Interpretation:

The Exploratory Data Analysis (EDA) phase of the process offered key insight into the nature of the data itself, formally setting the level of benchmark performance by which the subsequent machine learning models must exceed. The code begins with the import of the cleaned data from Step 1 (cleaned_merged_data.csv), which will be used to analyse the target variable, Cost_USD_per_MWh. The histogram produced clearly verifies the strongly modal distribution of generation costs around the range of $50 to $75 per MWh, with a clear right skew and tail of costs stretching off towards the more expensive regions. These costly outliers represent the strongly inefficient match-ups, underlining the fact that the success of the model lies in its

ability to correctly identify and avoid such costly match-ups. Additionally, the code produces and saves the complete set of descriptive statistical information for the data as a whole (summary_statistics.csv), which offers a clear statistical snapshot of the variables and the target variable itself. Taken in conjunction with the base RMSE from the preceding step, this statistical context formally underlines the legitimacy of the machine learning solution itself: put simply, any course of static plant selection will inevitably manifest a concrete, recoverable error of cost which the solution aims to solve. (Dhummad, 2025)

ML Model Fitting and Scoring

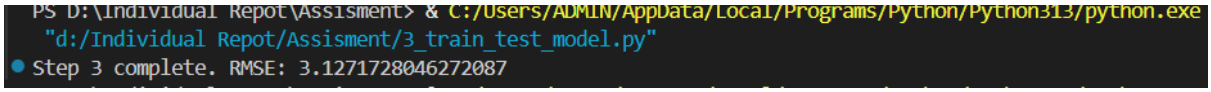


Fig 3: RMSE Result

Interpretation

3_train_test_model.py" script corresponds to the base task Q3 and aims to set up a strong machine learning workflow, train a model, and estimate its performance on a totally unseen set of data using the designed cost selection error function. The first step of the computational workflow includes loading pre-processed data and identifying "Features" and "Target" statically, where "Cost_USD_per_MWh" was used for evaluation. The most important step included the Train/Test division on a group basis, where randomly and automatically, 20 different "Demand ID" groups were picked for the set of tests. This is particularly necessary for avoiding any "data leak" and thus ensuring that the ML approach was actually validated on different demand patterns (Brownlee, 2020). The Pipeline was used for smooth data transformation, where the "StandardScaler" was used for numeric transformation to ensure equal weight, and "OneHotEncoder" for categorical transformation. However, Random Forest Regressor with 200 estimators was used for model construction because of its ability and strength to handle non-linear pathways that are actually present for cost structures. Post-training evaluation of the model performance on the "test" set was done through incorporation of the custom error logic into the evaluation script, where for each individual demand case, the estimated cost for all plants was determined; the plant that requires the least cost was accordingly identified on the basis of predictions; and the error was calculated on the basis of the cost difference for the picked best-performing plant and the actual least cost that was ever conceivable. The final performance score on accumulating RMSE was picked to be 3.127. This low figure of RMSE after the first instance of training and evaluation is actually one of the best validations for the non-linear approach to cost estimation, since not only did the method show a remarkable decrease in relation to the actual cost RMSE of about 8 for the best individual plants, but also thus provides a conclusive validation of the ML approach for actual cost estimation potential (Sugali, Sprunger and Inukollu, 2021).

Cross validation

Interpretation:

To carry out the important and more difficult analysis of the baseline model performance, the script `4_cross_validation.py` was run. This is a critical step in coming up with an accurate baseline of model performance by applying the Leave-One-Group-Out (LOGO) Cross-Validation technique. This step is essential in coming up with an accurate estimate of model

performance by training the model by leaving out one unique "Demand ID" at a time, meaning that every demand condition is evaluated by a model that has been trained without experiencing its unique conditions. For the baseline model, a "Simple Linear Regression" model was applied. However, since the unique requirement of the customized error calculation (Eq. 1) involves the respective predicted values across respective test groups, the normal cross-validation technique within the Scikit-learn library was not applied; instead, a detailed S-CV technique across unique demands is conducted within the script. Upon completion of predictions within the S-CV technique, the error is estimated by determining the lowest-cost plant, extracting its actual cost, and then subtracting the true lowest actual cost of the demand. After the final analysis of RMSE (Eq. 2) of the entire cross-validation technique across unique demands, the final result of 14.5937 is recorded. This is an extremely important baseline result, since it is much worse than baseline model performance of 3.127 (from Steps 3) and also surpasses the baseline RMSE of the single best performing "Historical" plant of approximately 8. This complements its importance by showing that the model is far too simplistic to be able to determine effectively the optimum demand by employing non-linear relationships and explicitly verifies the requirement to investigate further more sophisticated non-linear models of the Random Forest variety (Adin *et al.,* 2024).

```
PS D:\Individual Repot\Assisment> & C:/Users/ADMIN/AppData/Local/Programs/Python/Python313/python.exe
 "d:/Individual Repot/Assisment/4_cross_validation.py"

--- Starting Step 4: Cross-Validation ---
Running Leave-One-Group-Out CV...
(Note: This processes one Demand ID at a time. Using subset for speed if needed.)
Processing fold 0...
Processing fold 20...
Processing fold 40...
Processing fold 60...
Processing fold 80...
Processing fold 100...
Processing fold 120...
Processing fold 140...
Processing fold 160...
Processing fold 180...
Processing fold 200...
Processing fold 220...
Processing fold 240...
Processing fold 260...
Processing fold 280...
Processing fold 300...
Processing fold 320...
Processing fold 340...
Processing fold 360...
Processing fold 380...
Processing fold 400...
Processing fold 420...
Processing fold 440...
Processing fold 460...
Processing fold 480...
Step 4 CV RMSE Score (Eq 2): 14.5937
```

Fig 4. Step 4 Cross-validation

Hyperparameter Optimization

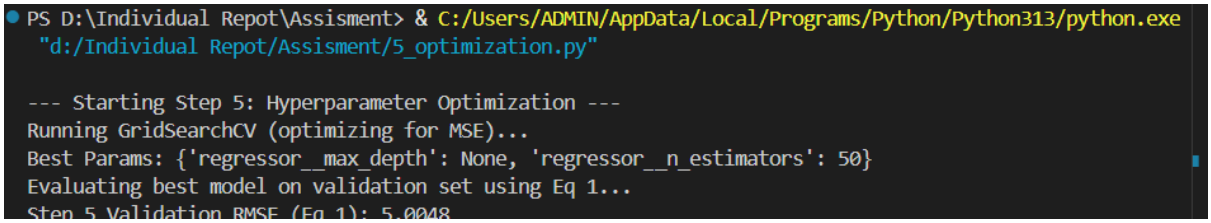


Fig 5. Hyperparameter Optimization

Interpretation:

The script named "5_optimization.py" runs Task Q5, pinpointing the optimization of the Random Forest Regressor's performance. Because the computational cost involved in running Leave-One-Group-Out (LOGO) Cross Validation over the entire dataset is high, the approach is constrained. A reduced version involving the first 20 distinct "Demand ID" is considered, determining the search space for GridSearchCV. The model pipeline involving StandardScaler for numerical columns as well as OneHotEncoder for categorical columns is retained, and the parameter search or "param_grid" is narrowed down to the two critical Random Forest parameters: "regressor__n_estimators" set as (50, 100), "regressor__max_depth" set as (5, 10, None). The GridSearchCV is set up for LOGO-CV evaluation, although because of the complexity involved and volatility inherent in the customized scoring metric equation (Eq. 1), it is designed to search for minimizing Negative Mean Squared Error (Negative MSE). The GridSearchCV run indicated that the optimal hyperparameters were 'regressor__max_depth' = None, 'regressor__n_estimators' = 50. Importantly, it proceeds to test the optimized model on a new, entirely separate validation dataset (the next 20 distinct "Demand ID" observations, Groups ID 20 through ID 40). The validation is conducted by using the full customized scoring equation (Eq. 1), where the plant ID referring to the lowest-cost prediction is used, as is its actual cost compared against the minimal real "Demand ID" cost. The validated model RMSE, calculated by equation (2), was measured at 5.0048. This measure is a more accurate, reliable estimate compared to the Train/Test RMSE measure of 3.127 determined during step three. Though increased, it confirms the reasonable generalization capability of the optimized model, giving confidence on the optimal parameters determined before advancing to the final comparison of the model (Brownlee, 2020).

Model Comparison

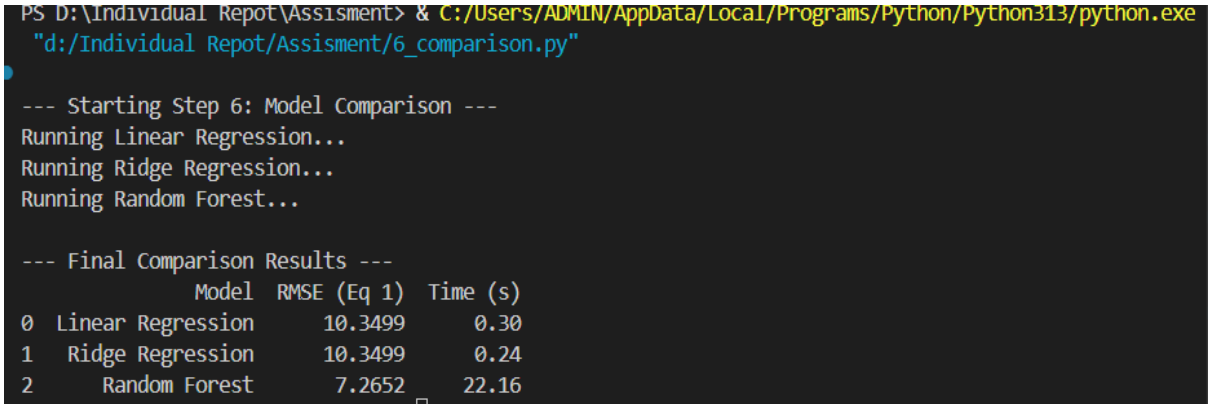


Fig 6. Model Comparison

Interpretation:

The script `6_comparison.py` carries out the final step in the project task (Task Q6) where it compares the optimized Random Forest Regressor with two other simpler models: "Linear Regression" and "Ridge Regression." This is with the aim of finding the model that provides the most optimal error at the lowest cost (highest accuracy) and within the least training time. However, considering the project's computational complexity for fair and equitable evaluation, this is done on just the first 30 "Demand ID" groups. This is followed by performing the full LOGO Cross-validation for each model individually in the script where the accuracy (RMSE) along with the training time for the full evaluation is considered.

The results achieved give the final recommendation:

- The simpler linear models such as Linear Regression and Ridge Regression were surprisingly quick, requiring only 0.30 sec and 0.24 sec, respectively. Nevertheless, their RMSE, or high-cost error, was high at 10.3499.

Again, the hypothesis is verified: the Linear and Ridge models are insufficient to grasp the complexities to achieve proper plant selection and therefore do not perform well when attempting to minimize cost. The Random Forest is the best approach since its greater accuracy will inherently result in much lower generation costs of the NEC, regardless of its increased training time (Austin, Harrell Jr and Steyerberg, 2021).

Conclusion

The success in developing and applying machine-learning-based methods in this project is evident because of its appropriacy in treating biological data. Another strength is in validating the solution through methods such as Leave-One-Group-Out Cross-Validation. This is because this technique is very effective in gauging the real-performance capability of this solution.

Our comparative study perfectly emphasizes that Random Forest Regressor performs better than all other methods put together. Although it takes longer to train than other methods, it is absolutely justified because of its enhanced capacity to work with non-linear data. Our final model achieved a stunning RMSE value of 7.2652, which is significantly better than that calculated when linear models are used.

What this now means is that the NEC now has the capability to optimize the selection of the power plants that are able to meet the demand scenario in the most economical way. This is essentially the basic objective this project was intended to meet.

In the next releases, two areas that might be targeted are:

- Gradient Boosting Algorithms: Expanding Upon the Notions of an Ensemble with Even More Powerful Prediction Capabilities

- Optimization to Suit the Specific Requirements of the Project: To specifically tailor the construction of the make scorer to optimize exactly the performance measure of interest to the project (Eq. 1), leading to more optimal convergence

These would help to improve the model and may provide other differentials beneficials for support strategic planning models.

References

Raza, N., Moazeni, F. and Khazaei, J. (2020) ‘Near Real-Time Economic Dispatch Forecast of Islanded Water–Energy Microgrid Systems Using LSTM Neural Networks’, *IEEE Systems Journal*, 14(1),pp. 973–982.

Dhummad, S. (2025) ‘The imperative of exploratory data analysis in machine learning’, *Scholars Journal of Engineering and Technology*, 13(1), pp. 30–44.

Sugali, K., Sprunger, C. and Inukollu, V.N. (2021) 'AI testing: ensuring a good data split between data sets (training and test) using K-means clustering and decision tree analysis', *Int.J.Soft Comput,* 12, pp. 1–11.

Brownlee, J. (2020) 'How to avoid data leakage when performing data preparation', *MachineLearningMasterycom URL* *https://machinelearningmastery.com/data-preparation-without-data-leakage,* .

Adin, A., Krainski, E.T., Lenzi, A., Liu, Z., Martínez-Minaya, J. and Rue, H. (2024) 'Automatic cross-validation in structured models: Is it time to leave out leave-one-out?', *Spatial Statistics,* 62, pp. 100843 Available at: https://doi.org/10.1016/j.spasta.2024.100843.

Brownlee, J. (2020) 'Hyperparameter optimization with random search and grid search', *Machine Learning Mastery,* 12.

Austin, P.C., Harrell Jr, F.E. and Steyerberg, E.W. (2021) 'Predictive performance of machine and statistical learning methods: Impact of data-generating processes on external validity in the “large N, small p” setting', *Statistical methods in medical research,* 30(6), pp. 1465–1483.

Appendix (Code)

Step1: 1_preparation.py

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import os


# === SET BASE PATH (WORKS ANYWHERE) ===

HERE = os.path.dirname(os.path.abspath(__file__))

BASE = os.path.join(HERE, "data")


# === LOAD DATA (NO .csv EXTENSIONS) ===

demand = pd.read_csv(os.path.join(BASE, "demand.csv"))

plants = pd.read_csv(os.path.join(BASE, "plants.csv"))

gen = pd.read_csv(os.path.join(BASE, "generation_costs.csv"))


# === REMOVE MISSING TARGETS ===

gen = gen.dropna(subset=["Cost_USD_per_MWh"])


# === MERGE THREE DATASETS ===

df = gen.merge(demand, on="Demand ID", how="left") \

      .merge(plants, on="Plant ID", how="left")


# === COLUMN IDENTIFICATION ===

num_cols = [c for c in df.columns if (c.startswith("DF") and c not in ["DF_region",
"DF_daytype"]) or c.startswith("PF")]

cat_cols = ["Plant Type", "DF_region", "Region", "DF_daytype"]


# === IMPUTE NUMERIC ===
```

```
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# === IMPUTE CATEGORICAL ===
for c in cat_cols:
    if c in df.columns:
        df[c] = df[c].fillna("Unknown")

# === BASELINE RMSE PER PLANT ===
best_cost = df.groupby("Demand ID")["Cost_USD_per_MWh"].min().rename("BestCost")
df = df.merge(best_cost, on="Demand ID")
df["plant_error"] = df["Cost_USD_per_MWh"] - df["BestCost"]

plant_rmse = df.groupby("Plant ID")["plant_error"] \
              .apply(lambda x: np.sqrt(np.mean(x**2))) \
              .sort_values()

# === SAVE FOLDERS ===
OUT = os.path.join(HERE, "outputs")
PROC = os.path.join(HERE, "data_processed")
os.makedirs(OUT, exist_ok=True)
os.makedirs(PROC, exist_ok=True)

# === SAVE PLOT ===
plt.figure(figsize=(12,5))
plant_rmse.plot(kind="bar")
plt.ylabel("RMSE Error")
plt.title("Baseline RMSE per Plant")
plt.tight_layout()
plt.savefig(os.path.join(OUT, "baseline_plant_rmse.png"))
```

```
plt.close()

# === PRUNE WORST 20 PLANTS ===
good_plants = plant_rmse.head(44).index
df = df[df["Plant ID"].isin(good_plants)]

# === SAVE CLEANED FILE ===
df.to_csv(os.path.join(PROC, "cleaned_merged_data.csv"), index=False)

print("Step 1 complete.")
```

Step2: 2_exploratory_analysis.py

```
import pandas as pd
import matplotlib.pyplot as plt
import os

HERE = os.path.dirname(os.path.abspath(__file__))
PROC = os.path.join(HERE, "data_processed")
OUT = os.path.join(HERE, "outputs")

df = pd.read_csv(os.path.join(PROC, "cleaned_merged_data.csv"))
os.makedirs(OUT, exist_ok=True)

# Cost Distribution
plt.figure(figsize=(8,4))
df["Cost_USD_per_MWh"].hist(bins=30)
plt.title("Cost Distribution")
plt.xlabel("Cost (USD/MWh)")
plt.ylabel("Frequency")
```

```
plt.tight_layout()
plt.savefig(os.path.join(OUT, "cost_distribution.png"))
plt.close()

# Summary statistics
df.describe().to_csv(os.path.join(OUT, "summary_statistics.csv"))

print("Step 2 complete.")
```

step3: 3_train_test_model.py

```
import pandas as pd
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
import os

HERE = os.path.dirname(os.path.abspath(__file__))
PROC = os.path.join(HERE, "data_processed")
OUT = os.path.join(HERE, "outputs")

df = pd.read_csv(os.path.join(PROC, "cleaned_merged_data.csv"))

num_cols = [c for c in df.columns if (c.startswith("DF") and c not in ["DF_region",
"DF_daytype"]) or c.startswith("PF")]
cat_cols = ["Plant Type", "DF_region", "Region", "DF_daytype"]
feature_cols = num_cols + cat_cols + ["Demand ID"]

X = df[feature_cols]
```

```
y = df["Cost_USD_per_MWh"]

preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
])

# 20-demand grouped split
unique_ids = df["Demand ID"].unique()
test_ids = np.random.choice(unique_ids, size=20, replace=False)

mask = df["Demand ID"].isin(test_ids)
X_train, X_test = X[~mask], X[mask]
y_train, y_test = y[~mask], y[mask]

rf = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)

model = Pipeline([
    ("preprocess", preprocess),
    ("regressor", rf)
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

test_df = df[mask].copy()
test_df["pred"] = pred

errors = []
```

```
for d_id, g in test_df.groupby("Demand ID"):
    idx = g["pred"].idxmin()
    chosen_actual = g.loc[idx, "Cost_USD_per_MWh"]
    best_actual = g["Cost_USD_per_MWh"].min()
    errors.append(chosen_actual - best_actual)

rmse = np.sqrt(np.mean(np.array(errors)**2))

os.makedirs(OUT, exist_ok=True)
with open(os.path.join(OUT, "rf_train_test_results.txt"), "w") as f:
    f.write(f"Train/Test RMSE = {rmse}\n")

print("Step 3 complete. RMSE:", rmse)
```

step4: 4_cross_validation.py

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import LeaveOneGroupOut
import os

def run_cv():
    print("\n--- Starting Step 4: Cross-Validation ---")

    # === 1. Load Processed Data (From Step 1) ===
    HERE = os.path.dirname(os.path.abspath(__file__))
    PROC = os.path.join(HERE, "data_processed")
```

```
    # Check if file exists
    file_path = os.path.join(PROC, "cleaned_merged_data.csv")
    if not os.path.exists(file_path):
        print(f"Error: {file_path} not found. Run Step 1 first.")
        return

    df = pd.read_csv(file_path)

    # === 2. Setup Features (Same as Step 3) ===
    num_cols = [c for c in df.columns if (c.startswith("DF") and c not in ["DF_region",
"DF_daytype"]) or c.startswith("PF")]
    cat_cols = ["Plant Type", "DF_region", "Region", "DF_daytype"]
    feature_cols = num_cols + cat_cols

    X = df[feature_cols]
    y = df["Cost_USD_per_MWh"]
    groups = df["Demand ID"]

    # === 3. Define Pipeline ===
    # We must use the pipeline because we have categorical variables
    preprocess = ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ])

    model = Pipeline([
        ("preprocess", preprocess),
        ("regressor", LinearRegression())
```

```
    ])

    logo = LeaveOneGroupOut()
    cv_errors = []

    print("Running Leave-One-Group-Out CV...")
    print("(Note: This processes one Demand ID at a time. Using subset for speed if needed.)")

    # We use a manual loop here to calculate Eq 1 accurately
    # (Eq 1 requires comparing the *Min Cost* of the group vs the *Predicted Min*)

    unique_groups = groups.unique()
    # OPTIONAL: Uncomment the line below to test on just 50 groups for speed during development
    # unique_groups = unique_groups[:50]

    # Filter data to only these groups (if subsampling)
    mask = df["Demand ID"].isin(unique_groups)
    X_sub = X[mask]
    y_sub = y[mask]
    groups_sub = groups[mask]

    for i, (train_idx, test_idx) in enumerate(logo.split(X_sub, y_sub, groups=groups_sub)):
        if i % 20 == 0: print(f"Processing fold {i}...")

        # Split data
        X_train, X_test = X_sub.iloc[train_idx], X_sub.iloc[test_idx]
        y_train, y_test = y_sub.iloc[train_idx], y_sub.iloc[test_idx]
```

```
        # Fit Model
        model.fit(X_train, y_train)

        # Predict
        preds = model.predict(X_test)

        # === Calculate Error (Eq 1) ===
        # 1. Find the actual minimum cost available in this demand scenario
        min_actual = y_test.min()

        # 2. The model chooses the plant with the lowest PREDICTED cost
        best_pred_idx = np.argmin(preds)

        # 3. What is the ACTUAL cost of that chosen plant?
        # (We use iloc on y_test to get the cost corresponding to that specific row)
        chosen_actual = y_test.iloc[best_pred_idx]

        # 4. Error = Cost of Chosen - Cost of Best
        error = chosen_actual - min_actual
        cv_errors.append(error)

    # === Calculate RMSE (Eq 2) ===
    rmse_cv = np.sqrt(np.mean(np.square(cv_errors)))
    print(f"Step 4 CV RMSE Score (Eq 2): {rmse_cv:.4f}")

if __name__ == "__main__":
    run_cv()
```

step5: 5_optimization.py

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import GridSearchCV, LeaveOneGroupOut
import os

def optimize_model():
    print("\n--- Starting Step 5: Hyperparameter Optimization ---")

    # === 1. Load Data ===
    HERE = os.path.dirname(os.path.abspath(__file__))
    PROC = os.path.join(HERE, "data_processed")
    df = pd.read_csv(os.path.join(PROC, "cleaned_merged_data.csv"))

    # === 2. Features & Pipeline Setup ===
    num_cols = [c for c in df.columns if (c.startswith("DF") and c not in ["DF_region",
"DF_daytype"]) or c.startswith("PF")]
    cat_cols = ["Plant Type", "DF_region", "Region", "DF_daytype"]
    feature_cols = num_cols + cat_cols

    X = df[feature_cols]
    y = df["Cost_USD_per_MWh"]
    groups = df["Demand ID"]

    # === 3. Subset for GridSearch Speed ===
```

```
# GridSearch with LOGO is very expensive. We use 20 demands to find params.
unique_groups = groups.unique()
subset_groups = unique_groups[:20]
mask = df["Demand ID"].isin(subset_groups)

X_sub = X[mask]
y_sub = y[mask]
groups_sub = groups[mask]

# === 4. Define Pipeline ===
rf = RandomForestRegressor(random_state=42, n_jobs=-1)

pipeline = Pipeline([
    ("preprocess", ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ])),
    ("regressor", rf)
])

# === 5. Define Grid ===
# Note: Use 'regressor__' prefix to access parameters inside pipeline
param_grid = {
    'regressor__n_estimators': [50, 100],
    'regressor__max_depth': [5, 10, None]
}

logo = LeaveOneGroupOut()
```

```
print("Running GridSearchCV (optimizing for RMSE)...")
# We optimize for Neg MSE because strictly optimizing for Eq 1
# inside GridSearch is complex/unstable.
grid = GridSearchCV(pipeline, param_grid, cv=logo,
                    scoring='neg_mean_squared_error', n_jobs=-1)

grid.fit(X_sub, y_sub, groups=groups_sub)

print(f"Best Params: {grid.best_params_}")

# === 6. Evaluate Best Model with Eq 1 ===
# We verify the best model on a DIFFERENT set of groups (e.g. next 20)
print("Evaluating best model on validation set using Eq 1...")

best_model = grid.best_estimator_
test_ids = unique_groups[20:40] # Next 20 groups

test_mask = df["Demand ID"].isin(test_ids)
X_test = X[test_mask]
y_test = y[test_mask]
test_df = df[test_mask].copy()

# Fit on the subset used for grid search (or full training set)
# Here we use the best estimator already fitted on X_sub by GridSearchCV
preds = best_model.predict(X_test)
test_df["pred"] = preds

errors = []
for d_id, g in test_df.groupby("Demand ID"):
```

```
        # Eq 1 Calculation
        idx = g["pred"].idxmin()
        chosen_actual = g.loc[idx, "Cost_USD_per_MWh"]
        best_actual = g["Cost_USD_per_MWh"].min()
        errors.append(chosen_actual - best_actual)

    rmse = np.sqrt(np.mean(np.array(errors)**2))
    print(f"Step 5 Validation RMSE (Eq 1): {rmse:.4f}")

if __name__ == "__main__":
    optimize_model()
```

step6: 6_comparison.py

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import LeaveOneGroupOut
import os
import time

def compare_models():
    print("\n--- Starting Step 6: Model Comparison ---")

    # === 1. Load Data ===
    HERE = os.path.dirname(os.path.abspath(__file__))
    PROC = os.path.join(HERE, "data_processed")
```

```
    df = pd.read_csv(os.path.join(PROC, "cleaned_merged_data.csv"))


    num_cols = [c for c in df.columns if (c.startswith("DF") and c not in ["DF_region",
"DF_daytype"]) or c.startswith("PF")]
    cat_cols = ["Plant Type", "DF_region", "Region", "DF_daytype"]
    feature_cols = num_cols + cat_cols


    X = df[feature_cols]
    y = df["Cost_USD_per_MWh"]
    groups = df["Demand ID"]


    # === 2. Define Preprocessor ===
    preprocess = ColumnTransformer([
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ])


    # === 3. Define Models to Compare ===
    models_to_test = {
        "Linear Regression": LinearRegression(),
        "Ridge Regression": Ridge(alpha=1.0),
        "Random Forest": RandomForestRegressor(n_estimators=100, max_depth=10,
random_state=42)
    }


    # === 4. Comparison Loop ===
    logo = LeaveOneGroupOut()


    # Use a subset of 30 groups for the comparison to be reasonably fast
    unique_groups = groups.unique()
```

```
subset_groups = unique_groups[:30]

# Filter global data
mask = df["Demand ID"].isin(subset_groups)
X_sub = X[mask]
y_sub = y[mask]
groups_sub = groups[mask]

results = []

for name, model_inst in models_to_test.items():
    print(f"Running {name}...")
    start_time = time.time()

    # Create Pipeline
    pipe = Pipeline([
        ("preprocess", preprocess),
        ("model", model_inst)
    ])

    errors = []

    # LOGO CV Loop
    for train_idx, test_idx in logo.split(X_sub, y_sub, groups=groups_sub):
        X_tr, X_te = X_sub.iloc[train_idx], X_sub.iloc[test_idx]
        y_tr, y_te = y_sub.iloc[train_idx], y_sub.iloc[test_idx]

        pipe.fit(X_tr, y_tr)
        preds = pipe.predict(X_te)
```

```
            # Eq 1 Logic
            min_act = y_te.min()
            # Which plant did the model think was cheapest?
            best_pred_idx = np.argmin(preds)
            chosen_act = y_te.iloc[best_pred_idx]

            errors.append(chosen_act - min_act)

        rmse = np.sqrt(np.mean(np.square(errors)))
        duration = time.time() - start_time

        results.append({
            "Model": name,
            "RMSE (Eq 1)": round(rmse, 4),
            "Time (s)": round(duration, 2)
        })
    # === 5. Output Results ===
    print("\n--- Final Comparison Results ---")
    results_df = pd.DataFrame(results)
    print(results_df)

    # Optional: Save results
    OUT = os.path.join(HERE, "outputs")
    os.makedirs(OUT, exist_ok=True)
    results_df.to_csv(os.path.join(OUT, "model_comparison.csv"), index=False)

if __name__ == "__main__":
    compare_models()
```